

CoreOps Week 4 Modules

Observability, Performance & Auto-Healing

Objective: Build a complete observability stack → understand performance bottlenecks → build auto-healing & scaling systems.



MONDAY: Prometheus, Grafana, kube-state-metrics

Critical Concepts

- What Prometheus actually does: pull-based scraping
 - Metrics types (Counter, Gauge, Histogram, Summary)
 - PromQL basics
 - kube-state-metrics vs node-exporter
 - Grafana dashboards: stat panel, graphs, alerts
-

WHAT TO PRACTICE

Lab 1: Install Prometheus + Grafana using kube-prometheus-stack (Helm)

helm repo add prometheus-community

<https://prometheus-community.github.io/helm-charts>

helm repo update

helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace

Lab 2: Explore Metrics

kubectl port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090 -n monitoring

Try PromQL:

- up
- node_cpu_seconds_total
- kube_pod_container_resource_requests_cpu_cores

Lab 3: Create a Grafana Dashboard

- Create a CPU graph for all nodes
 - Build a panel showing “Pods per Namespace”
-

WHERE TO STUDY

- Prometheus Docs → Basics + PromQL
- kube-state-metrics GitHub
- Grafana Academy (free)

INFRA THRONE MINDSET NOTES

- Metrics are truth; logs are storytelling.
 - 90% of outages show signals 10 minutes before failure in metrics.
 - Always monitor *control plane* separately from *workloads*.
-



TUESDAY: Loki + FluentBit Log Pipeline

Critical Concepts

- Difference: node logs vs container logs vs cluster logs
 - FluentBit architecture: Input → Filter → Output
 - Loki: labels, log volume, cardinality
 - Querying logs in Grafana (LogQL)
-

WHAT TO PRACTICE

Lab 1: Install Loki+Promtail

```
helm repo add grafana https://grafana.github.io/helm-charts
helm install loki grafana/loki-stack -n monitoring
```

Lab 2: Deploy a logging generator

```
kubectl run logger --image=busybox -- sh -c "while true; do echo hello; sleep 1; done"
```

Lab 3: Query logs in Grafana

```
{pod="logger"}
```

WHERE TO STUDY

- Loki Docs → LogQL queries
- FluentBit Filters & Kubernetes Inputs

INFRA THRONE MINDSET NOTES

- Logs without structure = pain. Add labels.
- Never enable high-cardinality labels. Costs explode first; cluster dies later.
- Logging pipelines must fail gracefully, not block workload execution.

WEDNESDAY: Distributed Tracing (OpenTelemetry)

Critical Concepts

- Why logs & metrics are not enough → latency breakdown
- Trace → Span → Attributes → Context propagation
- Service-to-service latency visualization
- Collector + Exporters (Tempo/Jaeger)

WHAT TO PRACTICE

Lab 1: Install OpenTelemetry Collector

kubectl apply -f

<https://raw.githubusercontent.com/open-telemetry/opentelemetry-operator/main/examples/collector.yaml>

Lab 2: Deploy a sample app with instrumentation

Use OpenTelemetry auto-instrumentation for a simple Python or Node app.

Lab 3: Visualize Traces in Grafana Tempo

Identify:

- Slowest span
- Service where latency originates

WHERE TO STUDY

- OpenTelemetry Docs: Collector + SDKs
- CNCF OTel YouTube tutorials

INFRA THRONE MINDSET NOTES

- Tracing fixes 20% of issues but saves 80% of debugging time.
- Use traces to solve “Where is the bottleneck?”, not “What crashed?”.
- Always tag spans with the deployment version.

THURSDAY: Autoscaling: HPA, VPA, KEDA

Critical Concepts

- HPA scaling signals (CPU, Memory, Custom metrics)
 - VPA types: Off, Auto, Recommendation
 - KEDA: event-driven autoscaling
 - Scaling patterns: burst, seasonal, queue-based, predictive
-

WHAT TO PRACTICE

Lab 1: Create an HPA

```
kubectl autoscale deployment nginx --cpu-percent=50 --min=1 --max=5
```

Lab 2: Load Test to Trigger Scaling

```
kubectl run loader --image=busybox -- sh -c "while true; do wget -q -O- http://nginx; done"
```

Lab 3: Install KEDA & Create ScaledObject

Example KEDA Redis scaler.

WHERE TO STUDY

- HPA Docs
- VPA GitHub
- KEDA documentation

INFRA THRONE MINDSET NOTES

- HPA is reactive → not ideal for sudden traffic spikes.
- VPA is powerful but dangerous in production without PDBs.
- Event-driven scaling saves cloud cost dramatically.

FRIDAY: Alerting, SLOs, Dashboards, SRE Practices

Critical Concepts

- SLI vs SLO vs SLA
- Error budgets
- Alert philosophy → “alert on symptoms, not causes”
- 4 golden signals: Latency, Traffic, Errors, Saturation
- How to build an actionable alert

WHAT TO PRACTICE

Lab 1: Create an Error Budget Burn Alert

Example rule:

- alert: HighErrorRate
expr: rate(http_request_errors_total[5m]) > 0.05
for: 2m
labels:
 severity: critical
annotations:
 summary: "High error rate detected"

Lab 2: Build an SLO Dashboard in Grafana

Panels for:

- Availability %
- Latency p90/p99
- Error budget remaining

Lab 3: Implement Alert Routing

Use Alertmanager:

- Critical → Slack
- Warning → Email or low-priority channel

WHERE TO STUDY

- Google SRE Book (SLOs chapter)
- Prometheus Alertmanager docs
- Grafana dashboard best practices

INFRA THRONE MINDSET NOTES

- Most companies fail due to poor alert design, not poor infra.
- Alerts must be actionable → if the on-call cannot fix it, don't alert it.
- Dashboards are not decoration; they narrate the system's health.

